Prompt de Arquitectura para Codex / Agente IA

Proyecto: VentasBot Paraguay (WhatsApp Cloud API + Infonet Bancard + React Admin)

Instrucciones para el Agente (Codex):

Eres un ingeniero de software senior. Tu tarea es inicializar, configurar y escribir todo el código para un proyecto full-stack (Backend en Node.js y Frontend en React con Vite). El proyecto usa SQLite para desarrollo local. Lee cuidadosamente la estructura de carpetas, ejecuta los comandos de configuración indicados y luego crea los archivos con el código exacto que se proporciona.

1. Estructura del Repositorio

Debes crear la siguiente estructura en el directorio raíz del proyecto:

```
/
├── backend/
│   ├── package.json
│   ├── .env
│   ├── prisma/
│   │   └── schema.prisma
│   └── src/
│       ├── server.js
│       ├── routes/
│       │   ├── whatsapp.routes.js
│       │   ├── bancard.routes.js
│       │   └── admin.routes.js
│       └── services/
│           ├── whatsapp.service.js
│           └── bancard.service.js
├── frontend/
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── App.jsx
│       └── index.css
```

2. Comandos de Inicialización (Ejecutar primero)

Agente, por favor ejecuta estos comandos en la terminal para preparar los entornos:

Para el Backend:

```
mkdir backend && cd backend
npm init -y
npm install express cors axios dotenv @prisma/client
npm install -D nodemon prisma
npx prisma init
```

Para el Frontend:

```
cd ..
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install tailwindcss @tailwindcss/vite
```

3. Código Fuente del Backend

backend/package.json

```
{
  "name": "ventasbot-backend",
  "version": "1.0.0",
  "description": "Backend VentasBot Paraguay con Bancard",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "dependencies": {
    "@prisma/client": "^5.13.0",
    "axios": "^1.6.8",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0",
    "prisma": "^5.13.0"
  }
```

```
}
```

backend/.env

```
PORT=3000

# Meta / WhatsApp API
WHATSAPP_TOKEN="tu_token_de_meta"
WHATSAPP_PHONE_NUMBER_ID="tu_id_de_numero"
VERIFY_TOKEN="ventasbot_secreto_123"

# Bancard vPOS
BANCARD_API_URL="https://vpos.infonet.com.py/vpos/api/0.3"
BANCARD_PUBLIC_KEY="tu_llave_publica"
BANCARD_PRIVATE_KEY="tu_llave_privada"
```

backend/prisma/schema.prisma

Agente, nota que estamos usando sqlite para entorno local. Después de crear este archivo, debes ejecutar npx prisma db push dentro de backend/.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = "file:./dev.db"
}

model Customer {
  id          String   @id @default(uuid())
  phone       String   @unique
  name        String?
  step        String   @default("START")
  activeOrderId String?
  orders      Order[]
}

model Product {
  id        String     @id @default(uuid())
  sku       String     @unique
```

```
  name       String
  price      Int
  stock      Int        @default(0)
  items      OrderItem[]
}

model Order {
  id             String   @id @default(uuid())
  bancardProcessId String?  @unique
  totalAmount     Int
  status          String   @default("PENDING")
  deliveryStatus  String   @default("PENDING")
  address         String?
  latitude        Float?
  longitude       Float?
  driverPhone     String?
  createdAt       DateTime @default(now())

  customerId      String
  customer        Customer @relation(fields: [customerId], references: [id])
  items           OrderItem[]
}

model OrderItem {
  id       String  @id @default(uuid())
  quantity  Int
  price    Int
  orderId   String
  productId String
  order    Order   @relation(fields: [orderId], references: [id])
  product   Product @relation(fields: [productId], references: [id])
}
```

backend/src/server.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = process.env.PORT || 3000;
```

```
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use('/api/whatsapp', require('./routes/whatsapp.routes'));
app.use('/api/bancard', require('./routes/bancard.routes'));
app.use('/api/admin', require('./routes/admin.routes'));

app.listen(PORT, () => {
  console.log(`🚀 Servidor VentasBot corriendo en http://localhost:${PORT}`);
});
```

backend/src/services/whatsapp.service.js

```
const axios = require('axios');

class WhatsAppService {
  constructor() {
    this.token = process.env.WHATSAPP_TOKEN;
    this.apiUrl =
`https://graph.facebook.com/v19.0/${process.env.WHATSAPP_PHONE_NUMBER_ID}/messages`;
  }

  async sendMessage(to, text) {
    try {
      await axios.post(this.apiUrl, {
        messaging_product: "whatsapp",
        to: to,
        type: "text",
        text: { body: text }
      }, { headers: this._getHeaders() });
    } catch (error) {
      console.error('Error WhatsApp Text:', error.response?.data || error.message);
    }
  }

  async sendRawMessage(payload) {
    try {
      await axios.post(this.apiUrl, payload, { headers: this._getHeaders() });
    } catch (error) {
      console.error('Error WhatsApp Raw:', error.response?.data || error.message);
    }
```

```
  }

  _getHeaders() {
    return {
      'Authorization': `Bearer ${this.token}`,
      'Content-Type': 'application/json'
    };
  }
}
module.exports = new WhatsAppService();
```

backend/src/services/bancard.service.js

```
const axios = require('axios');
const crypto = require('crypto');

class BancardService {
  constructor() {
    this.apiUrl = process.env.BANCARD_API_URL;
    this.publicKey = process.env.BANCARD_PUBLIC_KEY;
    this.privateKey = process.env.BANCARD_PRIVATE_KEY;
  }

  async createSingleBuy(orderId, amount) {
    const currency = 'PYG';
    const shopProcessId = orderId.slice(0, 30);

    const rawString = `${this.privateKey}${shopProcessId}${amount}${currency}`;
    const token = crypto.createHash('md5').update(rawString).digest('hex');

    const payload = {
      public_key: this.publicKey,
      operation: {
        token: token,
        shop_process_id: shopProcessId,
        amount: amount.toString(),
        currency: currency,
        description: `Orden #${shopProcessId.slice(-6)}`,
        return_url: 'https://wa.me/595XXXXXXXXX',
        cancel_url: 'https://wa.me/595XXXXXXXXX'
      }
    };
```

```
      try {
        const response = await axios.post(`${this.apiUrl}/single_buy`, payload);
        if (response.data.status === 'success') {
          return {
            url:
`https://vpos.infonet.com.py/payment/single_buy?process_id=${response.data.process_id}`,
            processId: response.data.process_id
          };
        }
        throw new Error('Error en Infonet');
      } catch (error) {
        console.error('Bancard Error:', error.response?.data || error.message);
        throw error;
      }
    }
}
module.exports = new BancardService();
```

backend/src/routes/whatsapp.routes.js

```
const express = require('express');
const router = express.Router();
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
const whatsappService = require('../services/whatsapp.service');
const bancardService = require('../services/bancard.service');

router.get('/webhook', (req, res) => {
  if (req.query['hub.verify_token'] === process.env.VERIFY_TOKEN) {
    res.send(req.query['hub.challenge']);
  } else {
    res.sendStatus(403);
  }
});

router.post('/webhook', async (req, res) => {
  const body = req.body;
  if (!body.entry?.[0]?.changes?.[0]?.value?.messages) return res.sendStatus(200);

  const message = body.entry[0].changes[0].value.messages[0];
  const phoneNumber = message.from;
```

```
    try {
      let customer = await prisma.customer.upsert({
        where: { phone: phoneNumber },
        update: {},
        create: { phone: phoneNumber }
      });

      if (message.type === 'order') {
        const catalogItems = message.order.product_items;
        let orderTotal = 0;
        const validItemsData = [];

        for (const item of catalogItems) {
          const product = await prisma.product.findUnique({ where: { sku:
item.product_retailer_id } });
          if (product) {
            orderTotal += product.price * item.quantity;
            validItemsData.push({ productId: product.id, quantity: item.quantity, price:
product.price });
          }
        }

        if (validItemsData.length === 0) return res.sendStatus(200);

        const newOrder = await prisma.order.create({
          data: {
            customerId: customer.id,
            totalAmount: orderTotal,
            items: { create: validItemsData }
          }
        });

        await prisma.customer.update({
          where: { id: customer.id },
          data: { step: 'PENDING_LOCATION', activeOrderId: newOrder.id }
        });

        const totalGs = new Intl.NumberFormat('es-PY').format(orderTotal);
        await whatsappService.sendMessage(phoneNumber, `🛒 Carrito recibido: Gs.
${totalGs}.\n\n📍 Para gestionar tu entrega, envíanos tu *Ubicación actual* (📎 -> Ubicación).`);
      }
      else if (customer.step === 'PENDING_LOCATION' && (message.type === 'location' ||
```

```
message.type === 'text')) {
        if (!customer.activeOrderId) return res.sendStatus(200);

        let address = message.type === 'location' ? (message.location.address || "Ubicación
GPS") : message.text.body;
        let lat = message.type === 'location' ? message.location.latitude : null;
        let lng = message.type === 'location' ? message.location.longitude : null;

        const order = await prisma.order.update({
          where: { id: customer.activeOrderId },
          data: { latitude: lat, longitude: lng, address: address }
        });

        const payment = await bancardService.createSingleBuy(order.id, order.totalAmount);

        await prisma.order.update({
          where: { id: order.id },
          data: { bancardProcessId: payment.processId }
        });

        await prisma.customer.update({
          where: { id: customer.id },
          data: { step: 'START', activeOrderId: null }
        });

        await whatsappService.sendMessage(phoneNumber, `💳 Toca el enlace para pagar de
forma segura con Infonet:\n\n${payment.url}`);
      }
      else if (message.type === 'interactive' && message.interactive.type === 'button_reply') {
        const btnId = message.interactive.button_reply.id;
        if (btnId.startsWith('DELIVERED_')) {
          const orderId = btnId.split('_')[1];
          const updatedOrder = await prisma.order.update({
            where: { id: orderId },
            data: { deliveryStatus: 'DELIVERED' },
            include: { customer: true }
          });
          await whatsappService.sendMessage(phoneNumber, "✅ Entrega confirmada.");
          await whatsappService.sendMessage(updatedOrder.customer.phone, "📦 ¡Tu pedido
ha sido entregado! Gracias por comprar.");
        }
      }
      else if (message.type === 'text' && customer.step === 'START') {
```

```
        await whatsappService.sendMessage(phoneNumber, "👋 ¡Hola! Escribe a este chat para abrir la tienda arriba y armar tu carrito.");
      }

    } catch (error) {
      console.error("Error Webhook:", error);
    }
    res.sendStatus(200);
});
module.exports = router;
```

backend/src/routes/bancard.routes.js

```
const express = require('express');
const router = express.Router();
const crypto = require('crypto');
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
const whatsappService = require('../services/whatsapp.service');

router.post('/webhook', express.json(), async (req, res) => {
    const { operation } = req.body;
    if (!operation) return res.status(400).send('Bad Request');

    try {
        const privateKey = process.env.BANCARD_PRIVATE_KEY;
        const rawString = `${privateKey}${operation.shop_process_id}pay${operation.amount}${operation.currency}`;
        const expectedToken = crypto.createHash('md5').update(rawString).digest('hex');

        if (operation.token !== expectedToken) {
            return res.status(401).send('Unauthorized');
        }

        const order = await prisma.order.findFirst({
            where: { id: { startsWith: operation.shop_process_id } },
            include: { customer: true }
        });

        if (order && order.status !== 'PAID') {
            await prisma.order.update({
                where: { id: order.id },
```

```
            data: { status: 'PAID', deliveryStatus: 'PREPARING' }
          });

          const totalGs = new Intl.NumberFormat('es-PY').format(order.totalAmount);
          await whatsappService.sendMessage(
            order.customer.phone,
            `*¡Pago Confirmado!* ✅\n\nRecibimos tu pago de Gs. ${totalGs} vía Bancard. Pronto
despacharemos tu pedido #${order.id.slice(-6)}.`
          );
        }
        res.status(200).json({ status: 'success' });
    } catch (error) {
        console.error("Error Bancard:", error);
        res.status(500).json({ status: 'error' });
    }
});
module.exports = router;
```

backend/src/routes/admin.routes.js

```
const express = require('express');
const router = express.Router();
const { PrismaClient } = require('@prisma/client');
const whatsappService = require('../services/whatsapp.service');
const prisma = new PrismaClient();

router.get('/orders', async (req, res) => {
    try {
        const orders = await prisma.order.findMany({
            include: { customer: true, items: { include: { product: true } } },
            orderBy: { createdAt: 'desc' }
        });
        res.json(orders);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

router.post('/orders/:id/dispatch', async (req, res) => {
    const { driverPhone } = req.body;
    try {
        const order = await prisma.order.update({
```

```
        where: { id: req.params.id },
        data: { deliveryStatus: 'ON_ROUTE', driverPhone },
        include: { customer: true }
      });

      await whatsappService.sendMessage(order.customer.phone, `🚚 *¡Tu pedido va en camino!*\nNuestro repartidor se dirige a tu ubicación.`);

      const mapsLink = `https://www.google.com/maps/search/?api=1&query=${order.latitude},${order.longitude}`;
      const driverMessage = {
        messaging_product: "whatsapp",
        to: driverPhone,
        type: "interactive",
        interactive: {
          type: "button",
          body: { text: `📦 *DESPACHO*\n\nOrden: #${order.id.slice(-6)}\nCliente: ${order.customer.phone}\nMapa: ${mapsLink}` },
          action: {
            buttons: [{ type: "reply", reply: { id: `DELIVERED_${order.id}`, title: "✅ Entregado" } }]
          }
        }
      };
      await whatsappService.sendRawMessage(driverMessage);

      res.json({ success: true });
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
});
module.exports = router;
```

4. Código Fuente del Frontend (Vite + React)

frontend/vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
```

```
    react(),
    tailwindcss(),
  ],
})
```

frontend/src/index.css

```
@import "tailwindcss";
```

frontend/src/App.jsx

```
import React, { useState, useEffect } from 'react';
import { Package } from 'lucide-react';
import axios from 'axios';

export default function App() {
  const [orders, setOrders] = useState([]);

  const fetchOrders = async () => {
    try {
      const res = await axios.get('http://localhost:3000/api/admin/orders');
      setOrders(res.data);
    } catch (error) {
      console.error("Error conectando al backend", error);
    }
  };

  useEffect(() => {
    fetchOrders();
    const interval = setInterval(fetchOrders, 10000);
    return () => clearInterval(interval);
  }, []);

  const handleDispatch = async (orderId) => {
    const driverPhone = prompt("Ingresa el WhatsApp del repartidor (ej. 595981...):");
    if (driverPhone) {
      await axios.post(`http://localhost:3000/api/admin/orders/${orderId}/dispatch`, { driverPhone
});
      fetchOrders();
      alert("¡Despachado con éxito!");
    }
  };
```

```
 return (
  <div className="min-h-screen bg-slate-50 flex font-sans text-slate-800">
   <aside className="w-64 bg-slate-900 text-white p-6 hidden md:block">
    <h1 className="text-2xl font-bold flex items-center gap-2"><Package/> VentasBot</h1>
    <nav className="mt-10 space-y-4">
     <div className="bg-slate-800 text-emerald-400 p-3 rounded-lg
font-medium">Logística</div>
    </nav>
   </aside>

   <main className="flex-1 p-8">
    <header className="mb-8">
     <h2 className="text-3xl font-bold">Pedidos por Despachar</h2>
    </header>

    <div className="bg-white rounded-xl shadow border border-slate-200
overflow-hidden">
     <table className="w-full text-left">
      <thead className="bg-slate-50 border-b border-slate-200 text-slate-500">
       <tr>
        <th className="p-4">ID</th>
        <th className="p-4">Cliente</th>
        <th className="p-4">Total</th>
        <th className="p-4">Estado</th>
        <th className="p-4">Acción</th>
       </tr>
      </thead>
      <tbody className="divide-y divide-slate-100">
       {orders.length === 0 && <tr><td colSpan="5" className="p-8 text-center
text-slate-400">Sin órdenes.</td></tr>}
       {orders.map(order => (
        <tr key={order.id} className="hover:bg-slate-50">
         <td className="p-4 font-mono font-bold">{order.id.slice(-6)}</td>
         <td className="p-4">{order.customer.phone}</td>
         <td className="p-4 text-emerald-600 font-bold">Gs.
{order.totalAmount.toLocaleString('es-PY')}</td>
         <td className="p-4">
          <span className="px-3 py-1 rounded-full text-xs font-bold
bg-slate-100">{order.deliveryStatus}</span>
         </td>
         <td className="p-4">
          {order.deliveryStatus === 'PREPARING' && order.status === 'PAID' && (
```

```
                <button onClick={() => handleDispatch(order.id)} className="bg-slate-900
text-white px-4 py-2 rounded-lg text-sm">
                  Despachar Moto
                </button>
              )}
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 </main>
</div>
);
}
```

Nota final para el Agente: Una vez termines de generar los archivos y correr el npx prisma db push, informa al usuario que puede iniciar el backend con npm run dev y el frontend con npm run dev.